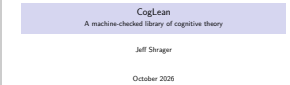
CogLean

A machine-checked library of cognitive theory

Jeff Shrager

October 2026

CogLean

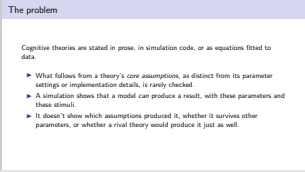


- This talk is about a way of stating cognitive theories so that what follows from them can be checked, compared, and accumulated.
- The vehicle is Lean, a programming language that is also a proof checker. You don't need to know Lean to follow this talk.
- The library is open source: github.com/jeffshrager/CogLean.

The problem

Cognitive theories are stated in prose, in simulation code, or as equations fitted to data.

- What follows from a theory's *core assumptions*, as distinct from its parameter settings or implementation details, is rarely checked.
- A simulation shows that a model *can* produce a result, with these parameters and these stimuli.
- It doesn't show which assumptions produced it, whether it survives other parameters, or whether a rival theory would produce it just as well.



- Most modelling evidence is a fit: one model, one set of parameters, compared with data.
- That's valuable, but it leaves three questions open: which assumptions did the work, is the result robust, and does it discriminate between theories?
- The rest of the talk is about answering those questions with proofs.

Three kinds of evidence a theory can give

One run	**Every run**	**Every model satisfying the postulates**
one model, one parameter setting	one model, all its executions	all models the theory allows
the theory *can* do it	that model *always* does it	the postulates *entail* it
simulation	model checking, sweeps	proof

CogLean is built around the third kind, and keeps the other two in the same language.

Three kinds of evidence a theory can give

Three kinds of evidence a theory can give

One run	Every run	Every model satisfying the postulates
one model, one parameter setting	one model, all its executions	all models the theory allows
the theory *can* do it	that model *always* does it	the postulates *entail* it
simulation	model checking, sweeps	proof

CogLean is built around the third kind, and keeps the other two in the same language.

- A theorem quantified over every model satisfying the postulates says which assumptions matter (the proof uses some and not others), what is robust, and whether a claim is possibility or necessity.
- Simulation is still essential: it explores, suggests conjectures, and fits data. Proof settles which conjectures follow from which assumptions. Experiments with people decide whether the assumptions are right.

Why Lean

- **One definition, run and proved.** The same definition of a memory can be *run* on an experiment's stimuli and *proved about* for all stimuli. The simulation and the theory can't drift apart.
- **Checked.** Every proof step is checked by a small trusted kernel.
- **General.** A theorem can quantify over all nets, all parameter values, all stimulus lists.
- **Cumulative.** Like Mathlib for mathematics: definitions made once, theorems imported rather than re-derived.

The guarantee is relative. Lean checks that a theorem follows from the definitions, not that the definitions capture the theory, still less that the theory is true of people.

Why Lean

- **One definition, run and proved.** The same definition of a memory can be *run* on an experiment's stimuli and *proved about* for all stimuli. The simulation and the theory can't drift apart.
- **Checked.** Every proof step is checked by a small trusted kernel.
- **General.** A theorem can quantify over all nets, all parameter values, all stimulus lists.
- **Cumulative.** Like Mathlib for mathematics: definitions made once, theorems imported rather than re-derived.

The guarantee is relative. Lean checks that a theorem follows from the definitions, not that the definitions capture the theory, still less that the theory is true of people.

- Mathlib is a single, shared, machine-checked library of mathematics. CogLean aims at the same thing for cognitive theory.
- The caveat on this slide matters and will come back: proofs are only as faithful as the formal definitions.

What is in the library

- **CogLib**: shared constructs (feature codes, exact probability distributions, discrimination nets, sampled retrieval, attention) and theory-neutral phenomena.
- **EPAM** (Feigenbaum): discrimination-net learning. Stimulus generalization, oscillation, retroactive inhibition, learning to criterion, the serial position effect: all proved. A published serial-anticipation run is reproduced (33–34 of 34 responses agree).
- **Strategy choice** (Siegler & Shrager): distributions of associations. Retrieval is used with probability $1 - (1 - R)^n$; and a finding: a more peaked distribution can *raise* overt strategy use.
- **Multi-digit addition**: the two combined, with 18 predictions neither makes alone.

What is in the library

- **CogLib**: shared constructs (feature codes, exact probability distributions, discrimination nets, sampled retrieval, attention) and theory-neutral phenomena.
- **EPAM** (Feigenbaum): discrimination-net learning. Stimulus generalization, oscillation, retroactive inhibition, learning to criterion, the serial position effect: all proved. A published serial-anticipation run is reproduced (33–34 of 34 responses agree).
- **Strategy choice** (Siegler & Shrager): distributions of associations. Retrieval is used with probability $1 - (1 - R)^n$, and a finding: a more peaked distribution can raise overt strategy use.
- **Multi-digit addition**: the two combined, with 18 predictions neither makes alone.

- Every theorem is fully proved, with no gaps, and uses only Lean's standard axioms.
- The peakedness result is a good example of what proof buys: an intuitive claim from the original chapter needs a condition the chapter doesn't state.

The generality problem

There are a great many specific models: EPAM, ACT-R, SAM, REM, MINERVA 2, TODAM, connectionist networks...

- Formalizing them one at a time doesn't scale.
- A pile of formalized models doesn't say what any two of them *disagree* about.
- And none of them is "true": unlike mathematics, there's no settled foundation to build on.

The generality problem

There are a great many specific models: EPAM, ACT-R, SAM, REM, MINERVA 2, TODAM, connectionist networks. . .

- Formalizing them one at a time doesn't scale.
- A pile of formalized models doesn't say what any two of them *disagree* about.
- And none of them is "true": unlike mathematics, there's no settled foundation to build on.

- This is the central design question. The answer on the next slides is to organize the library around laws, the way Mathlib organizes algebra.

Laws and families

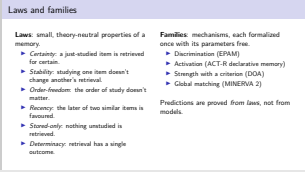
Laws: small, theory-neutral properties of a memory.

- *Certainty*: a just-studied item is retrieved for certain.
- *Stability*: studying one item doesn't change another's retrieval.
- *Order-freedom*: the order of study doesn't matter.
- *Recency*: the later of two similar items is favoured.
- *Stored-only*: nothing unstudied is retrieved.
- *Determinacy*: retrieval has a single outcome.

Families: mechanisms, each formalized once with its parameters free.

- Discrimination (EPAM)
- Activation (ACT-R declarative memory)
- Strength with a criterion (DOA)
- Global matching (MINERVA 2)

Predictions are proved *from laws*, not from models.



- Mathlib doesn't formalize every group; it proves theorems once per law (monoid, group, ring), and particular structures inherit them.
- Here, a prediction reads "any memory with stability predicts no interference with practiced items", not "EPAM predicts. . . ".

The theory matrix

	Certainty	Stability	Order-free	Recency	Stored-only	Determinate
Discrimination	**has**	**has**	lacks	lacks	**has**	**has**
Activation	lacks	lacks	lacks	**has**	**has**	lacks
Strength	lacks	lacks	**has**	lacks	**has**	lacks
Global matching	lacks	lacks	**has**	lacks	lacks	**has**

Every cell is a theorem: a general proof (**has**), or a counterexample (lacks). Every law separates some families, and every family has its own row.

The theory matrix

	Certainty	Stability	Order-free	Recency	Stored-only	Determinate
Discrimination	**has**	**has**	lacks	lacks	**has**	**has**
Activation	lacks	lacks	lacks	**has**	**has**	lacks
Strength	lacks	lacks	**has**	lacks	**has**	lacks
Global matching	lacks	lacks	**has**	lacks	lacks	**has**

Every cell is a theorem: a general proof (**has**), or a counterexample (lacks). Every law separates some families, and every family has its own row.

- "Has" cells hold for every parameter choice meeting stated conditions. "Lacks" cells are single counterexamples on small stimulus sets; where in its parameter space a family does have a law is further work.
- Global matching was added as a test. On the first four laws it looked exactly like strength memory, which forced the stored-only law: global matching's output is a blend, which can be a prototype never studied.
- The activation family here is ACT-R with decay 1 and temperature 1, where all its quantities are rational and exactly computable.

What is true here?

In mathematics, a proved theorem is true. Here there are only competing theories. What can a library build on?

Claim	Example	Status
A law entails a prediction	stability $\Rightarrow$ practiced items stay practiced	proved, permanent
A family has a law	discrimination nets have stability	proved, permanent
People have a law	people lack stability in recall	empirical, revisable

Which theory is right follows from the second and third rows together.

What is true here?

In mathematics, a proved theorem is true. Here there are only competing theories. What can a library build on?

Claim	Example	Status
A law entails a prediction	stability ⇒ practiced items stay practiced	proved, permanent
A family has a law	discrimination nets have stability	proved, permanent
People have a law	people lack stability in recall	empirical, revisable

Which theory is right follows from the second and third rows together.

- The conditional results don't expire. They stay true even if every family in the library is refuted.
- Mathlib is in the same position more often than it seems: it proves facts about Euclidean and non-Euclidean geometry alike, and leaves to physics which one space is.

A refutation spreads

- If people lack a law, every family that has it is refuted at once, and every prediction proved from it loses its support, in every theory that uses it.
- The matrix carries a judged evidence row, kept apart from the theorems.
- Each judgment states its **bridge**: the assumption connecting what experiments measure (reports, decisions) to what a law is about (what memory retrieves).

Example. People endorse a prototype they never saw. That does not show they *retrieved* it: a memory that returns a similar studied item also calls the prototype "old". The bridge is the missing piece.

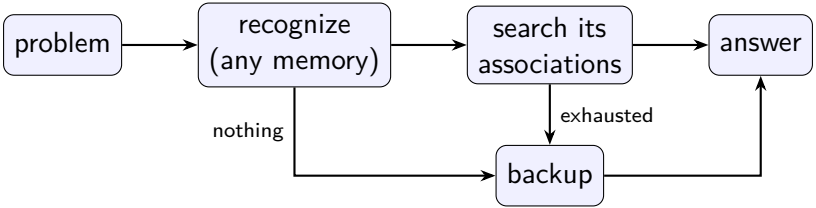
A refutation spreads

- If people lack a law, every family that has it is refuted at once, and every prediction proved from it loses its support, in every theory that uses it.
- The matrix carries a judged evidence row, kept apart from the theorems.
- Each judgment states its **bridge**: the assumption connecting what experiments measure (reports, decisions) to what a law is about (what memory retrieves).

Example. People endorse a prototype they never saw. That does not show they *retrieved* it: a memory that returns a similar studied item also calls the prototype "old". The bridge is the missing piece.

- As judged now: people lack certainty and order-freedom, have recency; stability and stored-only are mixed. Activation fits every settled entry; but each fit is a hypothesis, because the bridges are unformalized.
- The lesson: the next step for theory comparison is often not more data but an explicit response model.

Example 1: confident errors, inherited



- **Theorem**: a stranger taken for a familiar item with sharp associations is answered wrongly every time an answer is retrieved, with no more backup use than the familiar item.
- Proved once, for the composition. Inherited by multi-digit addition ($150 + 100$ taken for $100 + 100$) and by face naming (a stranger misnamed).
- From laws: any memory with determinacy, certainty and stability gives it. In the matrix, only discrimination.

Example 1: confident errors, inherited

Example 1: confident errors, inherited



- **Theorem**: a stranger taken for a familiar item with sharp associations is answered wrongly every time an answer is retrieved, with no more backup use than the familiar item.
- Proved once, for the composition. Inherited by multi-digit addition ($150 + 100$ taken for $100 + 100$) and by face naming (a stranger misnamed).
- From laws: any memory with determinacy, certainty and stability gives it. In the matrix, only discrimination.

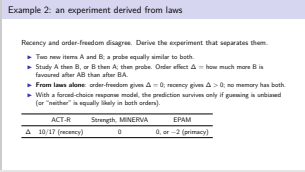
- The distributions-of-associations model alone can't produce these errors: there, errors and backup use come from the same distribution and rise together. Confusion gives a problem another problem's distribution.
- With ACT-R's memory, the errors are graded instead, and strangers say "don't know" more often.
- The evidence judges that people lack certainty, so a graded version of the theorem is what would explain confident errors in people.

Example 2: an experiment derived from laws

Recency and order-freedom disagree. Derive the experiment that separates them.

- Two new items A and B; a probe equally similar to both.
- Study A then B, or B then A; then probe. Order effect Δ = how much more B is favoured after AB than after BA.
- **From laws alone**: order-freedom gives $\Delta = 0$; recency gives $\Delta > 0$; no memory has both.
- With a forced-choice response model, the prediction survives only if guessing is unbiased (or "neither" is equally likely in both orders).

	ACT-R	Strength, MINERVA	EPAM
Δ	10/17 (recency)	0	0, or -2 (primacy)



- This reverses the usual workflow: start from a formal disagreement, derive the experiment, prove the separation.
- EPAM has neither law. Its order effect comes from the shape of the net, not from time: after a suitable prior item, whichever of A and B is learned first always wins.

Where this is weaker than mathematics

- **Definitions are contestable.** A family is one formalization of a theory. We check that link with blind simulations and community implementations.
- **Laws are idealizations** and presuppose a representation (binary codes, feature-count similarity).
- **Families are flexible.** The informative facts are parameter regions, not single counterexamples.
- **Evidence is never final.** The empirical row will change; the proved rows won't, and the matrix says at once what follows.

Where this is weaker than mathematics

- **Definitions are contestable.** A family is one formalization of a theory. We check that link with blind simulations and community implementations.
- **Laws are idealizations** and presuppose a representation (binary codes, feature-count similarity).
- **Families are flexible.** The informative facts are parameter regions, not single counterexamples.
- **Evidence is never final.** The empirical row will change; the proved rows won't, and the matrix says at once what follows.

- The blind simulation: a fresh agent wrote a Python simulator from a prose specification alone, against predictions preregistered before it existed. It agreed with Lean's exact distributions on every test, and exposed gaps in the formalization.

Next

- **Learning across trials**: feedback updates memory, so memory states become random; multi-digit learning sequences are the target.
- **Graded laws**, and the parameter regions of the "lacks" cells.
- **More families as tests**: REM, a connectionist memory, CHREST, ACT-R with blending.
- **Observational semantics**: decision and report components, so laws about retrieval predict what participants say.
- **Beyond memory**: strategy choice, then production systems.

Next

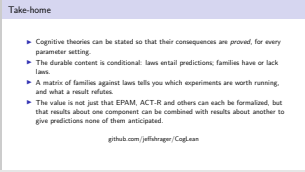
- **Learning across trials**: feedback updates memory, so memory states become random; multi-digit learning sequences are the target.
- **Graded laws**, and the parameter regions of the "lacks" cells.
- **More families as tests**: REM, a connectionist memory, CHREST, ACT-R with blending.
- **Observational semantics**: decision and report components, so laws about retrieval predict what participants say.
- **Beyond memory**: strategy choice, then production systems.

- The Common Model of Cognition is a useful checklist for which interfaces to add, not a foundation to impose.

Take-home

- Cognitive theories can be stated so that their consequences are *proved*, for every parameter setting.
- The durable content is conditional: laws entail predictions; families have or lack laws.
- A matrix of families against laws tells you which experiments are worth running, and what a result refutes.
- The value is not just that EPAM, ACT-R and others can each be formalized, but that results about one component can be combined with results about another to give predictions none of them anticipated.

github.com/jeffshrager/CogLean



- Two detailed talks follow up: one on recognition, retrieval and confident errors; one on the laws, the matrix and experiments derived from laws.